

详细答案版

1-热身题

$$1. \quad H(Y) = - \sum_{i=1}^2 p_i \log p_i = 1$$

$$p(Y=1) = p(Y=0) = \frac{1}{2}$$

$$I(Y; A) = H(Y) - H(Y|A)$$

$$\begin{aligned} &= 1 - \sum_{i=1}^n p_i H(Y|A=a_i) \\ 2. \quad &= 1 - \left[\frac{3}{8} \left(-\frac{1}{3} \cdot \log \frac{1}{3} - \frac{2}{3} \cdot \log \frac{2}{3} \right) + \frac{5}{8} \left(-\frac{3}{5} \cdot \log \frac{3}{5} - \frac{2}{5} \cdot \log \frac{2}{5} \right) \right] \\ &= 0.0488 \end{aligned}$$

$$I(Y; B) = H(Y) - H(Y|B)$$

$$\begin{aligned} &= 1 - \sum_{i=1}^n p_i H(Y|B=b_i) \\ 3. \quad &= 1 - \left[\frac{2}{8} \left(-\frac{2}{2} \cdot \log \frac{2}{2} \right) + \frac{3}{8} \left(-\frac{1}{3} \cdot \log \frac{1}{3} - \frac{2}{3} \cdot \log \frac{2}{3} \right) + \frac{3}{8} \left(-\frac{1}{3} \cdot \log \frac{1}{3} - \frac{2}{3} \cdot \log \frac{2}{3} \right) \right] \\ &= 0.3113 \end{aligned}$$

$$I(Y; C) = H(Y) - H(Y|C)$$

$$\begin{aligned} &= 1 - \sum_{i=1}^n p_i H(Y|C=c_i) \\ 4. \quad &= 1 - \left[\frac{3}{8} \left(-\frac{3}{3} \cdot \log \frac{3}{3} \right) + \frac{5}{8} \left(-\frac{4}{5} \cdot \log \frac{4}{5} - \frac{1}{5} \cdot \log \frac{1}{5} \right) \right] \\ &= 0.5488 \end{aligned}$$

$$5. C \quad I(Y; C) > I(Y; B) > I(Y; A)$$

6. A. 由于C的信息增益最大，所以选择C作为根节点。它将训练数据集D划分为两个子集D1, D2。对D1, D2重新计算 $I(D1, A)$, $I(D1, B)$, $I(D2, A)$, $I(D2, B)$

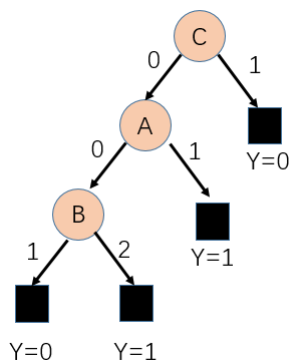
$$H(D1) = - \sum_{i=1}^2 p_i \log p_i = - \left(\frac{1}{5} \log \frac{1}{5} + \frac{4}{5} \log \frac{4}{5} \right)$$

$$H(D2) = - \left(\frac{3}{3} \log \frac{3}{3} \right) = 0$$

$$\begin{aligned}
 I(D1, A) &= H(D1) - H(D1|A) \\
 &= H(D1) - \left[\frac{2}{5} \left(-\frac{1}{2} \cdot \log \frac{1}{2} - \frac{1}{2} \cdot \log \frac{1}{2} \right) + \frac{3}{5} \left(-\frac{3}{3} \cdot \log \frac{3}{3} \right) \right] \\
 &= 0.3219
 \end{aligned}$$

$$\begin{aligned}
 I(D1, B) &= H(D1) - H(D1|B) \\
 &= H(D1) - \left[\frac{3}{5} \left(-\frac{1}{3} \cdot \log \frac{1}{3} - \frac{2}{3} \cdot \log \frac{2}{3} \right) + \frac{2}{5} \left(-\frac{2}{2} \cdot \log \frac{2}{2} \right) \right] \\
 &= 0.1710
 \end{aligned}$$

7.3, 树的深度的计算不包括叶节点



2 伪代码

```

# (a)
def predict(node, example):
    # base case: if the node is a leaf, return the majority vote
    if node.vote is not None:
        return node.vote

# (b)
'''
# 简洁版本
def predict(node, example):
    if node.vote is not None: # 叶子结点
        return node.vote
    elif example[node.attribute] == 1: # 非叶子结点
        return predict(node.left, example)
    else:
        return predict(node.right, example)
'''

# 完备版本complete "predict" function
def predict(node, example):
    '''
    node: root of the decision tree
    example: 要预测的样本, 根据decision tree预测样本的类别
    '''

```

```

'''
# 基础情况：如果节点是叶节点，则返回多数投票的预测
if node.vote is not None:
    return node.vote

# 递归步骤： 左1右0
# 如果节点的属性在示例中为1，则移动到左子节点
if example[node.attribute] == 1:
    # example可能有多个属性，先根据根节点的属性来看往左还是往右走
    # 额外的错误检查以确保存在左子节点
    if node.left is not None:
        return predict(node.left, example)
    else:
        raise ValueError(f"Missing branch at attribute
{node.attribute} for value 1")
# 如果节点的属性在示例中为0，则移动到右子节点
else: # example[node.attribute] == 0
    # 额外的错误检查以确保存在右子节点
    if node.right is not None:
        return predict(node.right, example)
    else:
        raise ValueError(f"Missing branch at attribute
{node.attribute} for value 0")
'''

```

这个函数首先检查当前节点是否是叶节点。如果是，那么它将返回这个叶节点保存的预测结果。如果不是，它将查看示例中当前节点属性的值，并根据该值向左或向右移动。在尝试向左或向右移动之前，它会检查相应的子节点是否存在，如果不存在，就会抛出一个错误。

3 实证问题

1.

heart:

0.49 0.4021

0.215 0.2784

0.215 0.3299

0.125 0.2577

education:

0.325 0.31

0.195 0.23

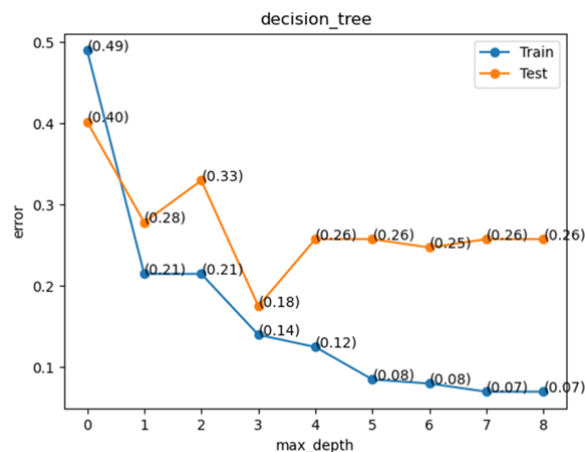
0.195 0.23

0.13 0.16

2. 见HW2/out_path_heart/heart.png

其实是不需要depth=9的，这里的9只是为了测试一下如果depth>num_attributions的情况是如何处理的

如果要修改的话，可以在decision_tree.py的line257处进行修改 `depths = list(range(train_data.shape[1]))`



3. D

在机器学习中，我们通常将数据划分为三个部分：训练集、验证集和测试集。各个部分的作用如下：

1. **训练集**：用于训练模型。模型的参数会根据训练集中的数据进行调整，以便最小化训练误差。
2. **验证集**：用于调整模型的超参数（例如决策树的最大深度）。通过在不同的超参数设置下训练模型，然后在验证集上评估性能，我们可以选择性能最好的超参数。避免使用训练集进行超参数优化的原因是，我们想要避免过拟合。过拟合意味着模型在训练数据上表现良好，但在未见过的数据上表现较差。
3. **测试集**：在模型训练和超参数调整完成后，最后使用测试集评估模型的性能。这可以给我们一个关于模型在现实世界数据上的泛化能力的无偏估计。

使用训练集优化最大深度可能会导致对训练数据过拟合，从而降低模型在未见过的数据上的泛化能力。因此，我们使用验证集来优化超参数，如决策树的最大深度。这样可以在某种程度上防止过拟合，并帮助我们选择在更广泛数据上表现良好的模型。

4. A

互信息即信息增益：

$$g(Y, A) = H(Y) - H(Y|A)$$

决策树学习中的信息增益等价于训练数据集中类与特征的互信息。互信息越大越好， $g(Y,A)$ 越大，说明 $H(Y|A)$ 越小， $H(Y|A)$ 越小说明根据属性A划分后熵值越低，越稳定，分类的清晰度越高，分类效果越好。

阈值越高，则深度越小，决策树包含的结点越少

实验部分

标签熵是一个常用于决策树和随机森林等机器学习算法的概念。它提供了关于某个数据集混乱程度的量度。标签熵的公式如下：

$$H(y) = \sum p(c_i) \log_2 \frac{1}{p(c_i)}$$

$$H(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

- $H(D)$ 是数据集 D 的熵。
- p_i 是第 i 个类别在数据集 D 中的频率。
- m 是类别的总数。

计算整个数据集（即在进行任何分裂之前）的标签熵时，会考虑所有可能的类别标签，并计算每个类别标签在数据集中的频率

inspection.py

计算标签熵和majority_vote的错误率，应该是在训练集上进行计算，训练集上才能训练majority_vote

small_inspect.txt

```
entropy: 1.0  
error: 0.5
```

heart_inspect.txt

```
entropy: 0.9997114417528099  
error: 0.49
```

education_inspect.txt

```
entropy: 0.9097361225311662  
error: 0.325
```

互信息：

选定某个特征，根据这个特征的分类把labels分为几堆，然后在对每一堆求熵

对所有的特征都求一遍互信息，然后找出来互信息最大的，作为根节点。

从余下的属性中继续求互信息，然后找出来下一个可以作为结点的属性

以此类推，构建决策树，同时要注意树的深度不能超过规定的深度，树的深度不包括叶节点，且从一开始计算。

例如下面的决策树：其深度为2

```
python decision_tree.py small_train.tsv small_test.tsv 2
small_2_train.txt small_2_test.txt small_2_metrics.txt
```

```
HW2 > ≡ small_depth_2.txt
1  [14 0/14 1]
2  | chest_pain = 0: [4 0/12 1]
3  | | thalassemia = 0: [3 0/4 1]
4  | | thalassemia = 1: [1 0/8 1]
5  | chest_pain = 1: [10 0/2 1]
6  | | thalassemia = 0: [7 0/0 1]
7  | | thalassemia = 1: [3 0/2 1]
8
```

```
python decision_tree.py education_train.tsv education_test.tsv 4
edu_4_train.txt edu_4_test.txt edu_4_metrics.txt > edu_depth_4.txt
```

HW2 > ≡ edu_depth_4.txt

```
1      [65 0/135 1]
2      | F = 0: [42 0/16 1]
3      | | M2 = 0: [27 0/3 1]
4      | | | M4 = 0: [22 0/0 1]
5      | | | M4 = 1: [5 0/3 1]
6      | | | | M5 = 0: [4 0/0 1]
7      | | | | M5 = 1: [1 0/3 1]
8      | | M2 = 1: [15 0/13 1]
9      | | | M4 = 0: [14 0/7 1]
10     | | | | P2 = 0: [6 0/0 1]
11     | | | | P2 = 1: [8 0/7 1]
12     | | | M4 = 1: [1 0/6 1]
13     | | | | M5 = 0: [1 0/1 1]
14     | | | | M5 = 1: [0 0/5 1]
15     | F = 1: [23 0/119 1]
16     | | M4 = 0: [21 0/63 1]
17     | | | M2 = 0: [18 0/26 1]
18     | | | | M5 = 0: [6 0/0 1]
19     | | | | M5 = 1: [12 0/26 1]
20     | | | M2 = 1: [3 0/37 1]
21     | | | | M5 = 0: [2 0/8 1]
22     | | | | M5 = 1: [1 0/29 1]
23     | | M4 = 1: [2 0/56 1]
24     | | | P1 = 0: [2 0/15 1]
25     | | | | M1 = 0: [2 0/7 1]
26     | | | | M1 = 1: [0 0/8 1]
27     | | | P1 = 1: [0 0/41 1]
28
```

```
python decision_tree.py heart_train.tsv heart_test.tsv 4
heart_4_train.txt heart_4_test.txt heart_4_metrics.txt >
heart_depth_4.txt
```

HW2 > ≡ heart_depth_4.txt

```
1  [102 0/98 1]
2  |  thalassemia = 0: [81 0/22 1]
3  | |  chest_pain = 0: [17 0/17 1]
4  | | |  fluoroscopy = 0: [14 0/4 1]
5  | | | |  angina = 0: [10 0/0 1]
6  | | | |  angina = 1: [4 0/4 1]
7  | | |  fluoroscopy = 1: [3 0/13 1]
8  | | | |  sex = 0: [3 0/2 1]
9  | | | |  sex = 1: [0 0/11 1]
10 | |  chest_pain = 1: [64 0/5 1]
11 | | |  flat_ST = 0: [14 0/3 1]
12 | | | |  abnormal_ecg = 0: [7 0/0 1]
13 | | | |  abnormal_ecg = 1: [7 0/3 1]
14 | | |  flat_ST = 1: [50 0/2 1]
15 | | | |  sex = 0: [25 0/0 1]
16 | | | |  sex = 1: [25 0/2 1]
17 |  thalassemia = 1: [21 0/76 1]
18 | |  fluoroscopy = 0: [17 0/23 1]
19 | | |  angina = 0: [13 0/8 1]
20 | | | |  flat_ST = 0: [4 0/6 1]
21 | | | |  flat_ST = 1: [9 0/2 1]
22 | | |  angina = 1: [4 0/15 1]
23 | | | |  sex = 0: [0 0/2 1]
24 | | | |  sex = 1: [4 0/13 1]
25 | |  fluoroscopy = 1: [4 0/53 1]
26 | | |  abnormal_ecg = 0: [4 0/22 1]
27 | | | |  flat_ST = 0: [1 0/15 1]
28 | | | |  flat_ST = 1: [3 0/7 1]
29 | | |  abnormal_ecg = 1: [0 0/31 1]
30
```


决策树构建好后，开始进行推理

对每个样本查看其属性，跟着决策树走，然后得到对应的分类结果。并计算对应的train error和test error

(1) 计算数据集 D 的经验熵 $H(D)$

$$H(D) = - \sum_{k=1}^K \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|}$$

(2) 计算特征 A 对数据集 D 的经验条件熵 $H(D|A)$

$$H(D|A) = \sum_{i=1}^n \frac{|D_i|}{|D|} H(D_i) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \sum_{k=1}^K \frac{|D_{ik}|}{|D_i|} \log_2 \frac{|D_{ik}|}{|D_i|}$$

(3) 计算信息增益

$$g(D, A) = H(D) - H(D|A)$$

解 首先计算经验熵 $H(D)$ 。

$$H(D) = -\frac{9}{15} \log_2 \frac{9}{15} - \frac{6}{15} \log_2 \frac{6}{15} = 0.971$$

然后计算各特征对数据集 D 的信息增益。分别以 A_1, A_2, A_3, A_4 表示年龄、工作、有自己的房子和信贷情况 4 个特征，则

(1)

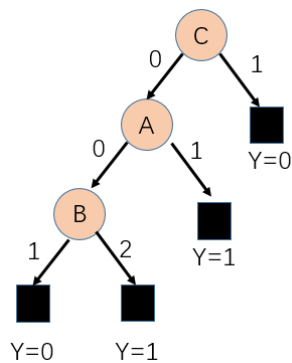
$$\begin{aligned} g(D, A_1) &= H(D) - \left[\frac{5}{15} H(D_1) + \frac{5}{15} H(D_2) + \frac{5}{15} H(D_3) \right] \\ &= 0.971 - \left[\frac{5}{15} \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) + \right. \\ &\quad \left. \frac{5}{15} \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) + \frac{5}{15} \left(-\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right) \right] \\ &= 0.971 - 0.888 = 0.083 \end{aligned}$$

这里 D_1, D_2, D_3 分别是 D 中 A_1 (年龄) 取值为青年、中年和老年的样本子集。类似地，

简洁答案(批改专用)

一.

1. 1
2. 0.0488
3. 0.3113
4. 0.5488
5. C
6. A
7. 3
8. 【4分】



二. 【6分】

```
# (1)
def predict(node, example):
    if node.vote != None:
        return node.vote

# (2)
def predict(node, example):
    if node.vote != None: # 叶子结点
        return node.vote
    if node.vote == None: # 非叶子结点
```

```

if example[node.attribute] == '1':
    return predict(node.left, example)
else:
    return predict(node.right, example)

```

三.

1. 【4分】

heart:

0.49 0.4021

0.215 0.2784

0.215 0.3299

0.125 0.2577

education:

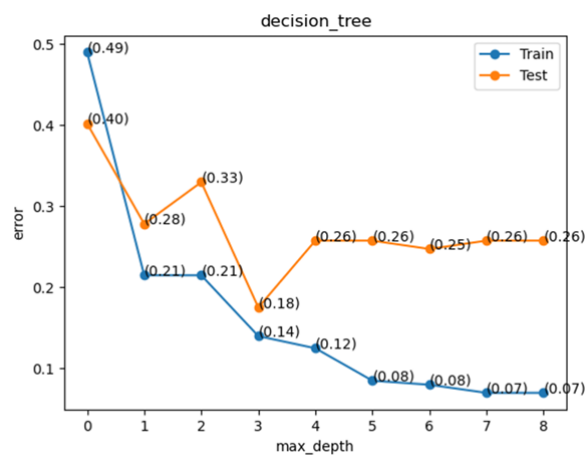
0.325 0.31

0.195 0.23

0.195 0.23

0.13 0.16

2. 【3分】



3. 【2分】 D

4. 【2分】 A

